SQL Database Project Report | Real Estate

Student Name – Aryan Sonker
Student ID -- 24141654

Introduction:

In the modern real estate industry, data plays a pivotal role in decision-making processes, from pricing strategies to market analysis and customer relationship management. Effective storage and management of this information requires a well-structured relational database capable of enforcing integrity constraints while supporting analytical queries. The real estate sector generates complex, multi-dimensional data involving properties, agents, transactions, and market behaviour typical of the UK housing market. The system is implemented using SQLite and populated programmatically using Python. Integrity constraints and foreign key relationships were applied to ensure structural consistency while preserving analytical flexibility.

Data Preparation and Methodology:

The dataset was programmatically generated using Python to ensure reproducibility, controlled randomness, and structural consistency. A fixed random seed(np.random.seed(42)) was used to ensure reproducibility, allowing the database to be regenerated with identical values if required.

Three datasets were programmatically generated for the database: 50 agent records, 1,000 property records, and 2,000 listing records. Attribute values were created within carefully defined ranges to ensure realism and consistency with the UK housing market. Property prices were generated between £80,000 and £1,200,000; bedrooms were limited to a range of 1–5, and property sizes varied between 400 and 3000 square feet. Agent ages were constrained between 22 and 65 to reflect plausible professional working ages, while year-built values ranged from 1960 to 2023 to represent a mix of older and modern housing stock.

Database Schema Design:

The database consists of three relational tables: Agents, Properties, and Listings. Each table was designed to represent a distinct real-world entity.

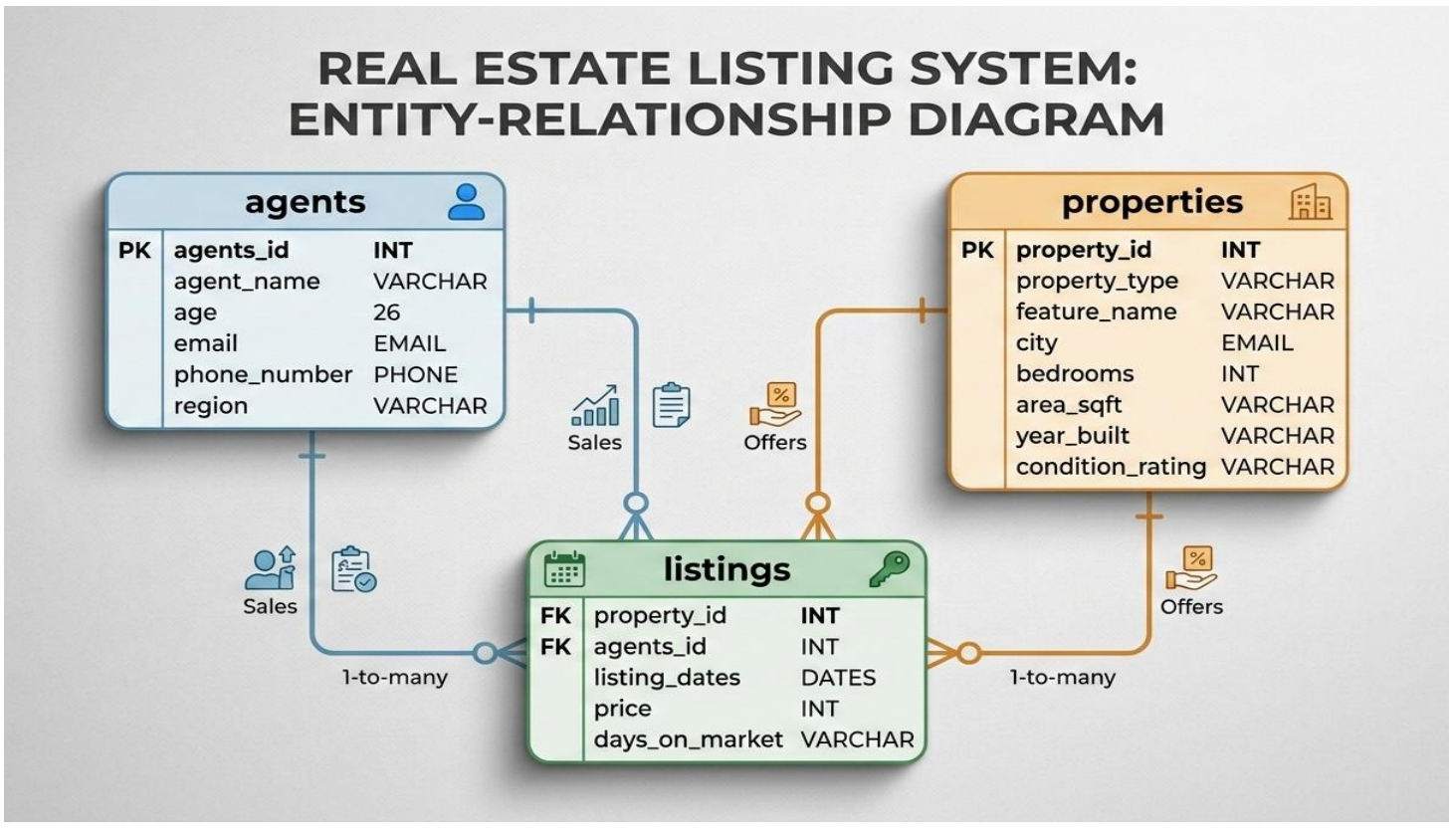

Table of Agents:

- The Agents table contains 50 real estate agents including agent ID, full name, age, email address, phone number, and assigned region (North, South, East, West).
- To imitate incomplete administrative records, approximately 10% of email addresses and phone numbers were deliberately left NULL.
- Age is ratio data (true zero and meaningful magnitude).
- Region is nominal (categorical).
- Agent name is nominal text data.
- Email addresses were generated to be unique to reflect real-world identification practices.
- A UNIQUE constraint prevents duplicate email entries.
- The primary key (agents_id) uniquely identifies each agent.

Table of Properties:

- The Properties table contains 1,000 properties including property ID, property type, feature name, city, number of bedrooms, area in square feet, year built, and condition rating.
- Property types (Apartment, House, Villa, Studio) are nominal categories.
- Feature name (Garage, Garden, Balcony, Pool, Fireplace) is nominal.
- City is nominal geographic data.
- Bedrooms and area_sqft are ratio variables (true zero, measurable magnitude).
- Year built is interval data (time-based but without a meaningful zero).
- Condition rating (1–5) is ordinal, representing increasing property quality.
- To simulate incomplete survey or inspection data, 5% of condition ratings were intentionally set to NULL.
- The primary key (property_id) uniquely identifies each property.

Table of Listings:

- The Listings table contains 2,000 property listings connecting agents and properties.
- Each listing includes property ID, agent ID, listing date, price (GBP), and days on market.
- Listing date is interval/temporal data (time-based).
- Price is ratio data (true zero and meaningful comparison).
- To simulate operational redundancy, 2% duplicate listing records were deliberately introduced.
- Days on market is ratio data representing listing duration.
- A composite primary key (property_id, agents_id, listing_dates) prevents an agent from listing the same property on the same date more than once.
- Foreign key constraints enforce referential integrity between listings, agents, and properties.

Data Types and Constraint Justification:

• **Table separation:** Agents, Properties, and Listings are logically divided to reduce redundancy and improve data integrity. The schema adheres to Third Normal Form (3NF), as non-key attributes are fully functionally dependent on their respective primary keys.

• **Primary and foreign keys:** Primary keys uniquely identify each agent and property, while foreign keys in the Listings table enforce referential integrity. This prevents invalid listings that reference non-existent agents or properties.

• **Composite primary key:** The composite key in the Listings table (property_id, agents_id, listing_dates) ensures that an agent cannot list the same property on the same date more than once, preventing duplicate transactional entries.

• **CHECK and UNIQUE constraints:** CHECK constraints enforce logical boundaries (e.g., positive price values, valid bedroom counts, realistic age ranges), while the UNIQUE constraint on agent email prevents duplicate identity records. These constraints enhance data validity and consistency.

• **Controlled missing and duplicate data:** Deliberately introduced NULL values and duplicate listing records simulate real-world data imperfections, increasing realism and enabling potential data validation and cleaning analysis.

Schema Relationships and Integrity:

The database enforces referential integrity using foreign key constraints:

- An agent may manage multiple property listings over time.
- A property may be listed multiple times, potentially by different agents or on different dates.
- The Listings table connects Agents and Properties through foreign key constraints.
- The composite primary key in the Listings table (property_id, agents_id, listing_dates) ensures that the same agent cannot list the same property on the same date more than once.
- Referential integrity is enforced to prevent listings from existing without valid associated agents and properties.

Name	Type	Schema
agents		CREATE TABLE agents (agents_id INTEGER PRIMARY KEY, agent_name TEXT NOT NULL, age INTEGER CHECK(age BETWEEN 18 AND 70), email TEXT UNIQUE, phone_number TEXT, region TEXT)
agents_id	INTEGER	"agents_id" INTEGER
agent_name	TEXT	"agent_name" TEXT NOT NULL
age	INTEGER	"age" INTEGER CHECK("age" BETWEEN 18 AND 70)
email	TEXT	"email" TEXT UNIQUE
phone_number	TEXT	"phone_number" TEXT
region	TEXT	"region" TEXT
listings		CREATE TABLE listings (property_id INTEGER, agents_id INTEGER, listing_dates TEXT, price REAL CHECK(price > 0), days_on_market INTEGER CHECK(days_on_market >= 0), PRIMARY KEY (property_id, agents_id, listing_dates), FOREIGN KEY (property_id) REFERENCES properties(property_id), FOREIGN KEY (agents_id) REFERENCES agents(agents_id))
property_id	INTEGER	"property_id" INTEGER
agents_id	INTEGER	"agents_id" INTEGER
listing_dates	TEXT	"listing_dates" TEXT
price	REAL	"price" REAL CHECK("price" > 0)
days_on_market	INTEGER	"days_on_market" INTEGER CHECK("days_on_market" >= 0)
properties		CREATE TABLE properties (property_id INTEGER PRIMARY KEY, property_type TEXT, feature_name TEXT, city TEXT, bedrooms INTEGER CHECK(bedrooms > 0), area_sqft INTEGER CHECK(area_sqft > 0), year_built INTEGER, condition_rating INTEGER CHECK(condition_rating BETWEEN 1 AND 5))
property_id	INTEGER	"property_id" INTEGER
property_type	TEXT	"property_type" TEXT
feature_name	TEXT	"feature_name" TEXT
city	TEXT	"city" TEXT
bedrooms	INTEGER	"bedrooms" INTEGER CHECK("bedrooms" > 0)
area_sqft	INTEGER	"area_sqft" INTEGER CHECK("area_sqft" > 0)
year_built	INTEGER	"year_built" INTEGER
condition_rating	INTEGER	"condition_rating" INTEGER CHECK("condition_rating" BETWEEN 1 AND 5)

Ethical Considerations and Data Privacy:

By using artificially generated data, this project avoids all ethical pitfalls associated with real personal data. No real individual's names, contact details, or financial information are stored. This approach aligns with the "data protection by design and by default" principle in GDPR Article 25. The database structure can be tested, analyzed, and shared without privacy concerns, demonstrating how ethical data practices can coexist with robust technical design.

- Secure storage and encryption of personal contact data.
- Role-based access control to prevent unauthorized data access.
- Data minimization principles.
- Anonymization where appropriate for analytical use.

Conclusion:

This project demonstrates the design and implementation of a realistic relational database for a real estate listing environment. The system incorporates multiple normalized tables, foreign key relationships, integrity constraints, and deliberately introduced data imperfections to reflect real-world operational conditions.

The project demonstrates practical application of database normalization, referential integrity enforcement, and controlled data simulation within a structured relational environment.

The database structure supports analytical tasks such as market trend evaluation, agent performance assessment, and property characteristic analysis. Through controlled data generation and constraint enforcement, the project illustrates both technical proficiency in relational database design and an understanding of practical data challenges.

Overall, the database achieves a balance between realism, structural rigor, and analytical flexibility, satisfying the requirements of robust relational modelling while remaining fully reproducible and ethically compliant.

Link for the Database:

https://github.com/REKNOS/SQL-Database